

AI Agent Architecture "Spectra" — BioSpecInfo

Field	Value
Project	BioSpecInfo — Spectra component (agentic AI copilot)
Author	Samuele Pio Provenzano
Thesis supervisor	Prof. Savino Longo — University of Bari Aldo Moro
Component	bsi-ai-hub.js — 5,876 lines, zero runtime dependencies
Type	Multi-provider conversational agent with client-side tool execution
Repository	samupropiol-ship-it/BioSpecInfo-v11
Documented version	Service Worker bsi-v140

1. Executive summary

Spectra is an **agent**, not a thin wrapper around a language model. The difference is measurable: the model does not answer quantitative questions from memory — it **invokes deterministic tools** that compute the result, and the control loop lets it **chain up to 10 calls**, verifying each step before answering.

The architecture addresses three problems that separate a usable agent from a demo:

1. **Grounding** — a fabricated chemical value can be dangerous. 32 tools cover computation, public databases and the application's own datasets; the system prompt explicitly forbids quoting numeric values from memory.
2. **Transparency** — the model's reasoning is streamed live and stored alongside the answer, together with the list of tools actually invoked. The agent's work is auditable after the fact.
3. **Provider portability** — a single tool registry is translated into the three function-calling formats in use today (OpenAI-compatible, Anthropic, Gemini), so the agent behaves identically across eleven model configurations, six of them free.

2. Architecture

2.1 Overview



There is no backend. API keys stay in the browser's `localStorage` and travel only to the selected provider: no data passes through BioSpecInfo servers, which do not exist.

2.2 Agentic loop

Each round: send the conversation history (last 40 turns) plus the system prompt and the schema of all 32 tools → receive the streamed response → if it contains tool calls, execute **all** of them → rebuild the assistant turn in the provider's native format → repeat.

Three exit conditions: no tool calls (final answer), `MAX_ROUNDS = 10` exhausted, or user interruption.

2.3 Provider-neutral tool registry

Tools are declared once in JSON Schema and translated at runtime:

Family	Required shape
Anthropic	<code>{name, description, input_schema}</code>
Gemini	<code>[{functionDeclarations: [...]}]</code>
OpenAI-compatible	<code>{type:"function", function:{...}}</code>

The same applies to messages: turns containing tool calls are serialised into each provider's native form and kept in a `_native` field, so a conversation stays coherent even when switching

models.

2.4 No hard-coded model names

A model name written into the code is a time bomb: it works until the provider retires that model, and then the application stops with a 404 the user cannot fix. It has happened twice in this project:

Provider	Error	When
Google	<code>models/gemini-1.5-flash</code> is not found for API version <code>v1beta</code>	removed from <code>v1beta</code>
Groq	The model <code>llama-3.3-70b-versatile</code> does not exist or you do not have access to it	deprecated 17/06/2026

On OpenRouter the problem is structural: free model ids change constantly, by design.

The fix is not to write the new name — that would be the same bomb with a longer fuse — but to **remove the name** and ask the API, which knows what exists *for that key*. This also resolves the ambiguity in Groq's message: "does not exist" **or** "you do not have access" are different cases, and a per-key listing tells them apart.

Every configuration with `modelliCandidati` starts at `model: null` and is resolved at runtime. The two families use different strategies because their names are different:

Gemini — version scoring. Names follow a regular scheme (`gemini-<major>.<minor>-<family>`), so they can be ordered: newest version first, `flash` family preferred, with penalties for experimental, `preview` and `-lite` variants; models without `streamGenerateContent` are discarded, as are non-conversational ones (embedding, image, native audio, Live API). When Gemini 3.0 ships it will be picked automatically.

OpenAI family — candidates in preference order. Here names are not comparable to each other (`openai/gpt-oss-120b` versus `qwen/qwen3.6-27b`) and automatic scoring would choose badly. `GET /models` is queried and the **first candidate that still exists** wins: the order of the list *is* the preference, and the listing serves to skip the ones that vanished. Only if no candidate survives does a generic score run over the available models — discarding transcription, speech, embedding and guard models, and preferring the larger model all else being equal.

Three fallback levels in both cases: model listing → candidate probing → first candidate, letting the real error from the generation call speak instead of inventing one.

The choice is cached for seven days per provider, keyed to a 32-bit FNV-1a fingerprint of the API key (different keys see different catalogues; the key itself is never duplicated). If the model is retired *while* cached, the 404 — or a 400 whose text mentions the model, as OpenRouter returns — invalidates the cache, re-resolves and retries **once**: a flag prevents an infinite loop when the new model fails too.

Anthropic is the deliberate exception: its models are paid, explicitly chosen by the user, and deprecated with long notice.

The six free services

Free quality is not all alike, and the difference matters: an 8-billion-parameter model does not hold up on a multi-step physical-chemistry problem.

Service	Models	The trade-off
GitHub Models	GPT-4.1, o4-mini, DeepSeek, Llama 4	The most capable free option, and it needs no new account: a token from the user's own GitHub is enough. In exchange, 10 requests/minute and 50 per day
NVIDIA NIM	DeepSeek R1, Qwen3 235B, Llama 4	Very large models, reasoning ones included. Credits run out and do not renew
Mistral	Large, Medium	~1 billion tokens per month. Requires phone verification and opting into data training
Groq	GPT-OSS 120B, Qwen3	The fastest, generous on volume
Google Gemini	Gemini Flash	Good context, generous
OpenRouter	automatic router	Picks among free models that support tool calling

None of these matches a paid frontier model on the hardest problems; GitHub Models and NVIDIA NIM come closest, at the cost of low request ceilings. That is why the list stays and the choice is the user's, rather than imposing one service.

2.5 The optional proxy — Spectra without a key

The API key lives in the user's browser: with no backend there is no alternative, and it is the limitation stated openly at the end of this document. Every visitor has to obtain their own free key and paste it in.

[proxy/spectra-proxy.js](#) is the way out, and it sits **outside** the page: a Cloudflare Worker (free plan) holding the keys as server-side secrets. When it is configured the browser carries no key at all, and anyone opening BioSpecInfo uses Spectra without entering anything.

It is not a plain forwarder:

- **The path passes through untouched.** The route is `/<provider>/<path>` and the rest is relayed as received: the proxy knows nothing about API shapes and keeps working when Spectra changes model or parameters.
- **Several keys per provider, with automatic takeover.** Each secret is a comma-separated list; on 429 (quota) or 401/403 (revoked key) it moves to the next one within the same request. Not on 400 — that is a bad request and would fail identically.
- **Streaming is never buffered:** the response body is passed straight through, otherwise answers would appear all at once at the end.
- **It is not an open relay.** Credentials arriving from the client (`Authorization`, `x-api-key`, `?key=`) are discarded and replaced: nobody can use the proxy to push a key of their own.

- **Three brakes** against abuse: an allowed-origins list, a per-IP limit and a daily cap.

On the Spectra side the graft is in one place. `GET /stato` reports which providers the proxy actually covers, so a model whose secret is missing is never offered as "no key needed"; `buildRequest` routes to the proxy **and omits authentication entirely**; `chiaveDaUsare()` returns a placeholder so the "missing key" guards do not block sending — a placeholder that never leaves the browser.

Both paths coexist: without `PROXY_URL` everything works as before, and for providers the proxy does not cover Spectra keeps using the local key.

2.6 Starting over

Everything Spectra remembers lives in `localStorage`: chats, history, API keys, persistent memory, review cards. A "clear history" button that removed only the conversations would not start over — reopening the app would restore the same state.

The delicate part is that keys exist in **two formats**: the current map (`bsi_api_keys`, one key per provider) and the single-key format of earlier versions (`bsi_api_key`). `getKeysMap()` migrates the latter into the former, so clearing only the map **brings the old key back** on the next access. Both must go — along with the resolved-model caches, which mean nothing without a key.

That is why the list of everything the application writes lives in one place, `DATI_CANCELLABILI`, split into five groups with a label and a description. The confirmation panel is built from it: it shows how many entries actually exist per group, disables the empty ones and requires a second explicit click. Only chats and keys are pre-selected; memory, review scheduling and the rest of the app's progress must be chosen deliberately, because they erase work unrelated to starting the conversation over.

Deliberately excluded: the PRO licence, the trial period and the device identity — data a user does not expect to lose by emptying a chat, and a licence must never be deleted by accident. The panel says so.

After deletion the in-process state is reset too — the list of providers the proxy covers, the resolved model per provider — otherwise it would stay valid until the page reloads, and Spectra would keep using a model chosen with a key that no longer exists.

2.7 Making free keys carry real load

Free tiers impose two different limits, and each needs a different answer.

The per-request ceiling. GitHub Models accepts 8,000 input tokens. Spectra's fixed cost is ~8,150 — 2,009 for the system prompt plus **6,128 for tool definitions alone** — so that service would not even start: it would fail before the user typed a word. The measurement came before the fix, and changed what the fix had to be.

The answer is not to trim history — it is to trim **the fixed cost**. Tools are paid on every turn; the conversation is the content. Of the two, the fixed cost must shrink first. `adattaAlBudget()` selects the tools **relevant** to the question instead of all 32, then shortens history with what remains.

Relevance is scored against a keyword map kept **outside** the tool registry: the words people use to *ask* for something are not the words a tool is *documented* with. The description of *astrofisica* talks about Wien and Stefan-Boltzmann; the user writes "what temperature is this nebula". Matching the question against descriptions alone picked *farmacocinetica* for an astrochemistry question — measured, not assumed. Matching is by stem (first five characters) because Italian inflects: "nebulosa" and "nebulose" must count as the same word.

Rate limits. A 429 on a free tier is almost always the *per-minute* limit, not an exhausted quota: waiting a few seconds clears it. The provider says how long in *Retry-After*, or in the error text ("*Please try again in 7.5s*"); when it says nothing, exponential backoff with a little jitter, so every open tab does not retry at the same instant. If the requested wait is absurd, it does not wait at all: better to say "quota exhausted" than to leave the user watching an hourglass for an hour.

Provider fallback. When a quota really is exhausted, the turn is redone on another service the user holds a key for. This is why several free keys together carry load that none of them carries alone.

Two non-negotiable constraints:

- It restarts from the **original** messages, not from half-finished history. Turns containing tool calls are stored in the previous provider's native format and cannot be handed to another. The turn's work is lost; a correct answer is gained.
- Fallback uses **free services only**. Silently moving to a paid one would spend the user's money without their say-so; if the chosen service was paid, it still gets the first attempt.

And it moves on **only** for exhausted quota. A 401 or a malformed request would repeat identically everywhere: trying all providers and then reporting a random last error would hide the real cause.

3. The 32 tools

Area	Tools
General computation	calcola, risolvi_equazione, analisi_dati
General chemistry	bilancia_equazione, stechiometria, massa_molecolare, converti_unita, costante_fisica
Physical chemistry	termodinamica, equilibrio_acido_base, cinetica, gas_e_soluzioni, elettrochimica
Structure & spectra	spettroscopia, quantistica_e_spettroscopia, cristallografia
Life sciences	biochimica, farmacocinetica, valuta_druglikeness
Physics	astrofisica, nucleare, statistica_inferenziale
External databases	cerca_pubchem (NIH), cerca_letteratura (PubMed), web search
Internal data	cerca_nel_database (9 datasets), cerca_molecola
App control	naviga_sezione (84 sections), apri_strumento (12 labs), stato_app
Memory	ricorda, ricordi
Animations	apri_animazione (6 reaction mechanisms)

3.1 Internal datasets exposed

297 synthesis reactions · 118 elements · 67 amino acids · 143 drugs · 63 pathologies · 39 retrosynthetic strategies · 36 drug interactions · 29 metabolic pathways · 29 redox potentials.

These are the datasets curated by the author for the application: the agent treats them as a primary source and is instructed to **report discrepancies** between the database and its own memory rather than silently smoothing them over.

4. Notable engineering decisions

This section documents the non-obvious choices and the reasoning behind them.

4.1 Expression engine without `eval()`

Problem. Serving arbitrary physical-chemistry computations requires an expression evaluator. `eval()` would solve it in a few lines.

Why it was rejected. The agent reads untrusted external content (PubChem results, PubMed abstracts, web pages). An injection in that content could induce the model to emit a hostile expression; with `eval()` that string would execute in the page context, **where the user's API key lives**.

Solution. Recursive-descent parser with a *closed* set of 27 functions and constants. It has no access to global scope: it can only do arithmetic. Validated against 10 escape attempts (`window.localStorage`, `constructor`, `this`, `fetch(...)`, calls to non-whitelisted functions) — all rejected, including when executed in a real browser.

4.2 Preserving reasoning blocks

On models with adaptive reasoning, `thinking` blocks are part of the assistant turn and must be **echoed back unchanged, cryptographic signature included**, on the next tool-use round. Dropping them causes the request to be rejected.

The implementation accumulates `thinking_delta` and `signature_delta` from the stream and reinserts them **first** in the reconstructed turn. Verified in integration tests: signature preserved, ordering correct.

4.3 Resuming paused turns (`pause_turn`)

Web search is a server-side tool. When its internal loop exhausts its iterations, the response ends with `stop_reason`: `"pause_turn"`.

The turn must be resumed by **sending the assistant turn back as-is, without adding any user message**: the server recognises the trailing `server_tool_use` block and continues from there. Appending a "continue" message would break the mechanism.

4.4 Per-model configuration, not global

The four Claude tiers do not accept the same parameters:

Model	<code>effort</code>	Web search	Notes
Fable 5.1	<code>xhigh</code>	yes (8)	refusal fallbacks; reasoning always on
Opus 5	<code>high</code>	yes (6)	default
Sonnet 5	<code>high</code>	yes (5)	
Haiku 4.5	unsupported	unsupported	sending them returns HTTP 400

On all recent models `temperature`, `top_p` and `budget_tokens` were removed from the API and their presence causes an error: none of the four sends them. Every parameter is therefore conditioned on the provider rather than set globally.

4.5 Multimodal input and material reading

The agent accepts images, PDFs and text files, several at once, treated as consecutive pages of a single document. Three constraints shaped the implementation:

- **Images are downscaled to 2000 px** and re-encoded as JPEG before sending. A phone photo drops from roughly 1 MB to 440 KB: without this step a single image would approach the request ceiling and multiply token cost. 2000 px is the minimum for reading small handwriting — at 1600 px fine script became illegible.
- **The PDF page ceiling depends on the model**, it is not a constant: 600 pages for models with a 1M-token window, 100 for 200K ones. Page counting is done by reading `/Type` `/Page`

objects from the file bytes, without libraries.

- **History stores a 160 px thumbnail**, not the image that was sent. Storing the large one (440 KB each) filled `localStorage` within a dozen photos, and once the quota is exceeded every subsequent write fails silently.

Six quick actions turn the material into a study artefact (summary, concept map, outline, flashcards, exam questions, transcription) and two translate it. For transcriptions the agent is instructed to write `[illegible]` rather than guess: an invented chemical term is worse than a declared gap.

4.6 In-house graph renderer, no CDN

Concept maps are produced by the model as Mermaid diagrams and drawn by a renderer written into the application (~190 lines): longest-path layering from the roots with cycle breaking, barycentre reordering over three passes to reduce edge crossings, SVG output with Bézier curves.

Not loading Mermaid from a CDN follows from the app's offline-first nature: a map must be drawable without a network. Unsupported syntaxes are recognised and left as a readable code block instead of being drawn badly.

4.7 Voice command with wake word

User-activated continuous listening: the phrase "Hey Spectra" followed by a command sends it automatically. Two non-obvious details:

- the browser's speech recognition **stops on its own** after a few seconds of silence and must be restarted, otherwise listening dies at the first pause;
- **all transcription alternatives** are examined, not just the first, and the most common phonetic variants are accepted: Italian recognition renders "Spectra" in many different ways.

Denying microphone permission stops the loop instead of retrying forever. The word "spettroscopia", extremely frequent in this domain, was verified as non-triggering.

4.8 Guard against non-finite results

A systematic audit revealed that, given legitimate degenerate inputs (zero wavelength, zero volume, zero half-life, a transition between identical levels), eight solvers returned `ok: true` with `Infinity` or `NaN` among their fields.

This is the most insidious failure in this context: an exception is noticed, a formally valid but meaningless value is reported to the user as correct. The fix is a single check at the point where all results pass through: if a numeric field is not finite, the result becomes an explicit error naming the fields and stating the probable cause. It also covers tools added in the future.

4.9 Graceful degradation

- **Models without reliable function calling** (some free tiers): on the first tool-related error the request is retried once in text-only mode instead of failing.

- **Idle timeout (45 s)**, reset on every byte received: a long answer is never truncated, but a dead connection is reported immediately with an explicit message.
- **localStorage quota**: history is capped (30 conversations, 100 messages) with progressive pruning. Without this cap, exceeding the quota made every subsequent write fail *silently*, **including saving the API key**.
- **Classifier refusals** (`stop_reason`: "refusal", HTTP 200): detected and surfaced, instead of appearing as an empty answer.

5. Verification

Verification was carried out at three levels, with recorded outcomes.

5.1 Numeric correctness — against reference values

Area	Case	Expected	Obtained
Balancing	$\text{KMnO}_4 + \text{HCl} \rightarrow \dots$	2:16:2:2:8:5	✓ exact
Molecular mass	$\text{K}_4[\text{Fe}(\text{CN})_6]$ (nested)	368.35	368.345
Molecular mass	$\text{CuSO}_4 \cdot 5\text{H}_2\text{O}$ (hydrate)	249.68	249.677
Equilibrium	acetic acid 0.1 M, K_a $1.8 \cdot 10^{-5}$	pH 2.874	2.875
Thermodynamics	ΔG ($\text{N}_2\text{O}_4/\text{NO}_2$) at 298 K	+4.79 kJ/mol	✓
Kinetics	E_a from $k(300 \text{ K})$, $k(320 \text{ K})$	55.3 kJ/mol	55.33
Real gases	van der Waals CO_2	−10 % deviation	−10.13 %
Spectroscopy	bromine M+2	97.3 %	✓
Biochemistry	glycylglycine	132.12 Da	✓
Pharmacokinetics	$t_{1/2}$ from V_d 50 L, Cl 5 L/h	6.93 h	✓
Astrophysics	solar emission peak (Wien)	501.5 nm	501.52
Astrophysics	Earth escape velocity	11.19 km/s	11.186
Nuclear	^{56}Fe binding energy	8.79 MeV/nucleon	8.790
Statistics	critical t, 95 %, $df = 4$	2.7764	✓
Crystallography	copper density (fcc)	8.96 g/cm ³	8.935

P-values are not tabulated: they are computed with the regularised incomplete beta function (Lentz continued fraction) and the incomplete gamma function.

5.2 Robustness — cases that must fail

Verified as **rejected**: unbalanceable equations, formulas with unbalanced parentheses or non-existent elements, species absent from the equation, invalid database types, and the 10 injection attempts against the expression engine.

5.3 Integration — real browser

Automated tests with headless Chromium across all 14 application pages: no JavaScript errors, no missing resources. All 84 sections of [index.html](#) and the 18 tabs of the Astrochemistry module were traversed one by one. Request bodies were inspected for the four Claude tiers, and a full cycle with web search, turn suspension and resumption was simulated.

6. Quantitative summary

Metric	Value
Component size	5,876 lines
Tools	32
Model configurations	11 (6 free)
Scientific areas covered	13
Records in exposed internal datasets	over 800
Max agentic loop rounds	10
History sent to the model	40 turns
Runtime dependencies	none

7. Stated limitations

Documented for technical honesty; these are deliberate choices, not omissions.

- **The API key lives in the browser** — unless the proxy is deployed. In the default configuration the key never leaves the device, but it is accessible to anyone with access to that browser, and every user must obtain their own. The Worker in section 2.5 removes this limitation by moving keys to a server, at the cost of one component to maintain outside the page: a distribution choice, not a requirement.
- **Network tools depend on service availability.** PubChem and PubMed are queried directly from the browser; when unreachable, the tool returns an explicit error and the agent is instructed to say so rather than substituting an estimate.
- **Solvers apply simplified models** where the literature offers more refined ones (e.g. Woodward-Fieser for UV, or ATP yields using average P/O ratios). Each tool states the method used in its own result.
- **Verification is functional and numeric**, not formal: there is no proof of solver correctness, but a set of reference cases drawn from the teaching literature.

Companion document to the BioSpecInfo technical dossier. See also: [01-Software-Architecture-Document.md](#), [02-Verification-Validation-Report.md](#), [03-Security-Privacy-Compliance.md](#).